2026睿抗机器人开发者大赛--大模型及智能体应用样题

> **场景**：智慧图书馆AI服务平台 ｜ **时长**：240 分钟 ｜ **满分**：100 分 ｜ **形式**：在线答题 · 填空计分

说明

1. 本卷全部题目以**在线填空**作答：答案填写在答题页对应空号处，空号 N 与题面 `__【N】__` 的 N 一一对应。
2. **运行环境**：Python 3.11（conda 环境 `raicom`）；运行所需的依赖、对话模型与中文嵌入模型**均已就位**。凡涉及安装、下载的命令仅作**知识性考核**，不需要实际执行。
3. **工程构成**：**后端工程**为 `library-ai-platform/`（Python/FastAPI 四层服务）——本卷全部题目的作答对象均出自该工程及其随卷附件；**前端工程**为 `library-ai-web/`（Vue 3 + Vite 控制台，按路径前缀代理四层），仅作架构背景，**不参与作答**。
4. 英文命令、函数名、标识符、常量与枚举**不区分大小写**；可省略正则的 r 前缀与引号。
5. 中文术语须按题面括号内提示**完整填写**。
6. 标注「本题指定」的参数值为唯一正确答案，不得替换为其它合理值。
7. 代码填空**仅填写空缺部分本身**，不得附带等号、引号、括号或语句分隔符。
8. **空号**：全卷空号 **1→94** 连续编号，题面代码框内与正文内的空均写作 `__【N】__`（N 为该空的全卷空号；一题多空时按空号从小到大作答）。

本项目后端工程工作根目录为 `library-ai-platform/`。

第一部分　项目背景

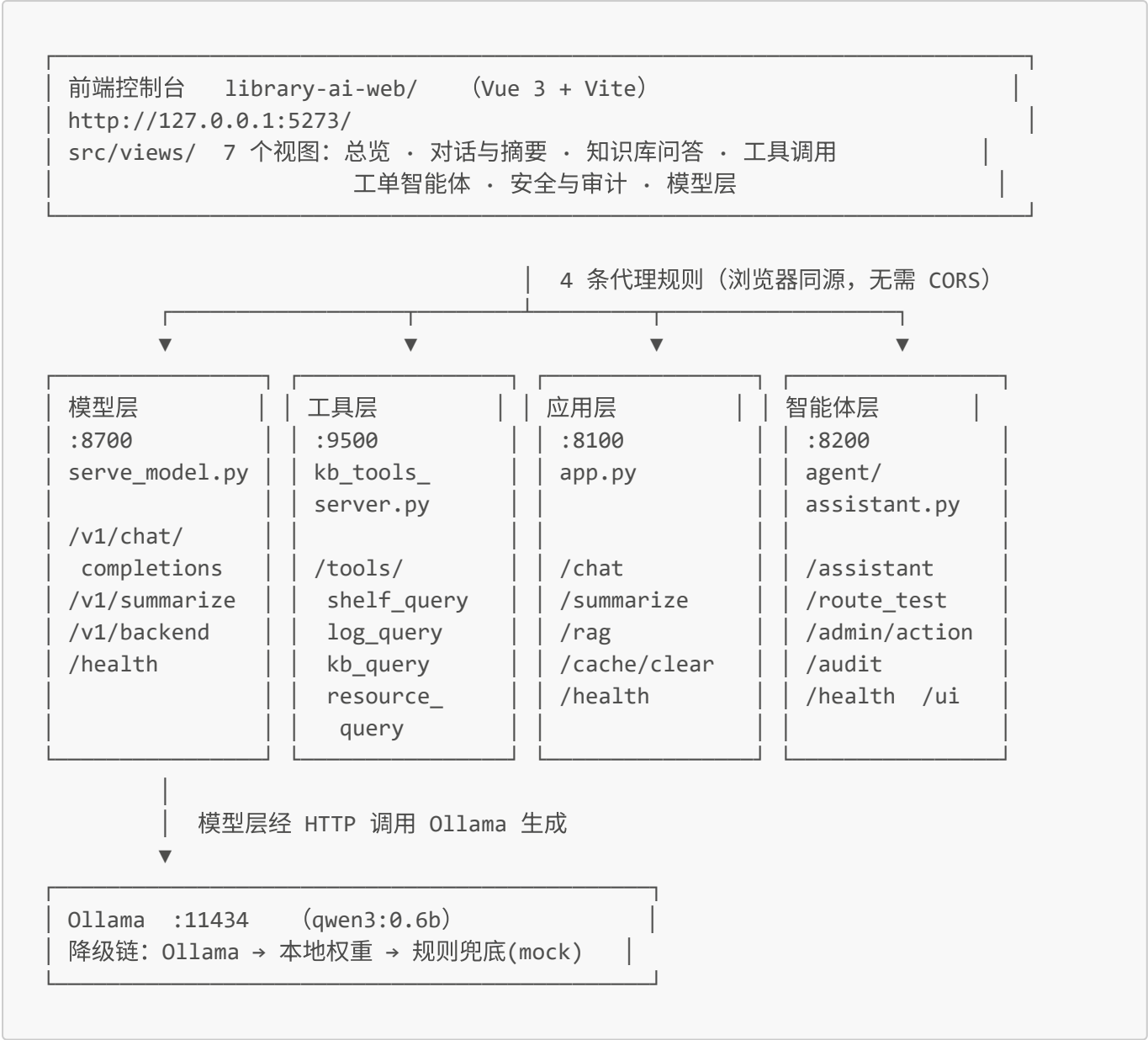
某高校图书馆日均入馆两万余人次，总服务台每天要回答上千个重复咨询：“借阅卡丢了怎么补办？”“研讨间怎么约？”“校外怎么访问数据库？”与此同时，馆内自助借还机、空调、门禁的故障长期靠电话报障、人工分派，处置慢、留痕难。

技术部决定启动 **智慧图书馆AI服务平台**，按软件工程全流程分三期建设，恰好对应本卷三个模块：

- **模块一（模型层）**：部署 Qwen3-0.6B 推理服务，并基于三年咨询日志完成数据清洗、微调样本构建与小规模 LoRA 微调；
- **模块二（应用层）**：开发馆内服务助手，交付多轮对话、通知摘要、政策知识增强问答（RAG），并完成缓存与降级优化；
- **模块三（智能体层）**：以纯 Python 实现设备报修工单多智能体系统，叠加统一入口路由、敏感指令拦截、权限校验与操作审计。

系统架构总览

平台按**四层架构**实现，每层是独立的 Python 进程、各自监听一个端口，层与层之间只通过 HTTP 调用；浏览器侧由前端统一代理，因而不存在跨域问题。



调用方向：**智能体层** → 应用层(:8100) + 工具层(:9500)；**应用层** → 模型层(:8700)；**模型层** → Ollama(:11434)。四层没有任何聚合层，前端按路径前缀分别代理。

层	端口	实现文件	对外接口
模型层	**8700**	`serve_model.py`	本地模型推理（OpenAI 风格 `/v1/chat/completions`）等
工具层	**9500**	`kb_tools_server.py`	4 个 GET 接口：座位/设备状态、服务日志、知识库检索、电子资源并发
应用层	**8100**	`app.py`	多轮对话 `/chat`、通知摘要 `/summarize`、知识增强问答 `/rag`
智能体层	**8200**	`agent/assistant.py`	工单统一入口 `/assistant`、操作审计 `/audit` 等

后端工程的模块构成（各文件的职责与依赖关系）：

模块	文件	职责	依赖
公共库	`common.py`	路径、端口与全项目公共常量（各层共用）	—
公共库	`llm_runtime.py`	模型加载、对话模板、文本生成、向量嵌入	Ollama / 本地权重
模型层 :8700	`serve_model.py`	本地模型推理服务（OpenAI 风格接口）	`llm_runtime`
工具层 :9500	`kb_tools_server.py`	4 个工具接口（座位/设备、服务日志、知识库、电子资源）	—
应用层 :8100	`app.py`	多轮对话、通知摘要、知识增强问答（含缓存与降级）	`llm_runtime` · `rag/rag.py`
应用层	`cli.py`	命令行客户端	应用层 HTTP 接口
应用层	`rag/rag.py` · `build_index.py`	政策文档切分、检索与索引构建	嵌入模型 · faiss
智能体层 :8200	`agent/assistant.py`	统一入口：路由、敏感拦截、权限校验、操作审计	`agent/agent.py` · 应用层 · 工具层
智能体层	`agent/agent.py` · `agent/tools.py`	工单流程编排、工具注册与调用封装	工具层
随卷附件	`env_check.py` · `perf_check.py` · `eval_compare.py` 等	环境校验、性能对比、效果对比	—

技术栈

类别	组件	版本 / 说明
运行时	Python（conda 环境 `raicom`）	3.11
深度学习框架	PyTorch（CPU 版）	由官方 CPU 索引源安装
模型加载与微调	transformers / peft / datasets	4.51.3 / 0.15.2 / 3.5.0
模型运行时	Ollama	HTTP `:11434`，对话模型 `qwen3:0.6b`
向量化与检索	sentence-transformers / faiss-cpu	3.4.1 / 1.10.0
Web 服务	fastapi / uvicorn / requests	0.115.12 / 0.34.2 / 2.32.3

> 前端工程 `library-ai-web/`（Vue 3 + Vite）仅作架构背景，不参与作答；浏览器经它按路径前缀代理四层。

第二部分　实操竞赛题目

模块一：模型层：环境部署与 LoRA 微调

任务1：开发环境核验与依赖检查

题目1.1　创建并激活 Conda 环境，请写出对应命令。

```
# 1. 创建名为 raicom 的 Python 环境
conda create -n __【1】__ python=3.11 -y
# 2. 激活该环境
conda activate __【2】__
```

raicom

raicom

题目1.2　安装 CPU 版 PyTorch，请写出安装命令中应使用的索引源（知识性考核，不需要实际执行）。

```
pip install torch --index-url __【3】__
```

https://download.pytorch.org/whl/cpu

题目1.3　附件 requirements.txt 中缺少一个 LoRA 微调必需的关键依赖（缺少它训练会报 ModuleNotFoundError），请补全依赖名。

```
transformers==4.51.3
peft==0.15.2
modelscope==1.25.0
datasets==3.5.0
sentence-transformers==3.4.1
faiss-cpu==1.10.0
fastapi==0.115.12
uvicorn==0.34.2
requests==2.32.3
# 请补充缺少的依赖：
__【4】__
```

accelerate

题目1.4　运行环境校验脚本（附件 `env_check.py`），其日志会写入固定的落盘目录（相对于后端工程根目录）。请补全该目录。

```
pip install -r requirements.txt
python env_check.py | tee __【5】__/env_verify.log
```

submisson/m1

任务2：大模型服务部署与接口配置

实际使用ollama,这里对应ollama失效时的本地权重配置

ID格式:组织/模型名

题目2.1　比赛指定模型已就位（目录 `./models/Qwen3-0.6B`）。请写出应使用的模型仓库 ID。

```
modelscope download --model __【6】__ --local_dir ./models/Qwen3-0.6B
```

Qwen/Qwen3-0.6B

题目2.2　编写推理服务 serve_model.py，将模型部署为 RESTful 服务。请补充代码空缺（生成参数按本题指定：长度上限 256、关闭采样）。

```
import torch, uvicorn
from fastapi import FastAPI, Request
from transformers import AutoModelForCausalLM, AutoTokenizer

MODEL_PATH = "./models/Qwen3-0.6B"
tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)
model = AutoModelForCausalLM.__【7】__(MODEL_PATH, torch_dtype=torch.float32)
model.eval()
app = FastAPI()

@app.__【8】__("/v1/chat/completions")
async def chat(req: Request):
    messages = (await req.json())["messages"]
    prompt = tokenizer.__【9】__(
        messages, tokenize=False, add_generation_prompt=True)
    inputs = tokenizer(prompt, return_tensors="pt")
    out = model.generate(**inputs, max_new_tokens=__【10】__, do_sample=__【11】__)
    ...

uvicorn.run(app, host="0.0.0.0", port=8700)
```

frompretrained
post
req: Request
就是声明：这个函数收进来的参数
req 代表"这一次进来的 HTTP
apply_chat_templation
256
False

题目2.3　用 curl 完成接口连通性测试。请补充端口号与预期 HTTP 状态码。

8700

```
curl -s -X POST http://127.0.0.1:__【12】__/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"messages":[{"role":"user","content":"你好"}]}' \
  -o submission/m1/api_test_result.json -w "%{http_code}"      # 预期状态码 __【13】__
```

200

题目2.4　Qwen3 支持"思考模式"。本卷要求在 CPU 上关闭思考模式。请写出关闭时所传的参数值（填 True 或 False）。

```
prompt = tokenizer.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True,
    enable_thinking=__【14】__)          # CPU 环境关闭思考以提速
```

False

任务3：咨询日志清洗与微调样本构建

题目3.1　对手机号做脱敏处理。请补充正则与占位符。

```
import re                                          r"1[3-9]/d{8}"
PHONE_RE = re.compile(__【15】__)        # 中国大陆手机号：11 位，号段 1[3-9] 开头
record["phone"] = PHONE_RE.sub(__【16】__, record["phone"])
                                                   "****"
```

题目3.2　过滤测试数据与空回答记录，并按 9:1 划分训练集与验证集。请补充两处空缺。

这个空要根据比赛具体文件做,看看数据格式,字典键值对

```
valid = [r for r in data
         if not r["session_id"].startswith(__【17】__)
         and all(t["a"].strip() for t in r["turns"])]
n_train = int(len(valid) * __【18】__)      # 9:1 划分
```

r['turns']能看出来是包含的所有的对话,t是取出来的一轮,t['a']就是模型的回答,

"session_id"表示这是第几个对话窗口,'turns'就是这个窗口对话轮次

0.9

题目3.3　将有效记录转换为指令微调标准格式（OpenAI/Qwen 对话格式）。请补充顶层字段名。

```
{"__【19】__": [{"role":"user","content":"..."},
                {"role":"assistant","content":"..."}]}
```

messages

任务4：LoRA 小规模微调与效果验证

题目4.1　使用 PEFT 对 Qwen3-0.6B 做 LoRA 微调。请补充配置代码（秩按本题指定为 8）。

```
from peft import LoraConfig, get_peft_model          8
lora_cfg = __【20】__(task_type="CAUSAL_LM", r=__【21】__, lora_alpha=16,
                     target_modules=["q_proj","k_proj","v_proj","o_proj"],
                     lora_dropout=0.05)
model = __【22】__(model, lora_cfg)
```

LoraConfig

get_peft_model

题目4.2　为控制 CPU 训练时间，请补充训练参数（只训练 1 轮、在 CPU 上训练）。

```
training_args = TrainingArguments(
    output_dir="submission/m1/finetune",
    num_train_epochs=__【23】__,
    per_device_train_batch_size=2,
    learning_rate=2e-4,
    logging_steps=10,
    use_cpu=__【24】__)
```

1

True

模块二：应用层：馆内服务助手开发

任务5：对话与摘要能力实现

题目5.1　实现多轮对话接口 /chat。请补充代码空缺（生成参数按本题指定：长度上限 256、关闭采样）。

```
HISTORY = {}

@app.__【25】__("/chat")
async def chat(req: Request):
    body = await req.json()
    sid, text = body["session_id"], body["text"]
    history = HISTORY.__【26】__(sid, [])      # 会话历史不存在则创建
    history.append({"role": "user", "content": text})
    prompt = tokenizer.apply_chat_template(history, tokenize=False,
             add_generation_prompt=True, enable_thinking=False)
    inputs = tokenizer(prompt, return_tensors="pt")
    out = model.generate(**inputs, max_new_tokens=__【27】__, do_sample=__【28】
__)
    answer = tokenizer.decode(out[0][inputs["input_ids"].shape[1]:],
                              skip_special_tokens=True)
    history.append({"role": __【29】__, "content": answer})
    return {"session_id": sid, "reply": answer}
```

post　setdefault　256　False　'assistant'

题目5.2　实现通知摘要接口 /summarize。摘要指令「请生成100字以内摘要」应放在__【30】__角色的消息中（填英文角色名）；其生成长度上限（本题指定）为__【31】__。

```
messages = [
    {"role": "__【30】__", "content": "请生成100字以内摘要"},
    {"role": "user", "content": announcement},
]
out = model.generate(**inputs, max_new_tokens=__【31】__, do_sample=False)
```

'system'　128

题目5.3　编写命令行联调脚本 cli.py。请补充两个内置/库函数名。

```
while True:
    text = __【32】__("请输入问题（输入q退出）: ")
    if text == "q": break
    r = requests.__【33】__("http://127.0.0.1:8100/chat",
        json={"session_id": "cli", "text": text}).json()
    print(r["reply"])
```

input　post

题目5.4　max_new_tokens 的计数单位是__【34】__（英文单词单数，含义为“词元/令牌”；提示：它并不等同于中文字数）；若要在返回前统计摘要的字符数，可用的 Python 内置函数是__【35】__。

```
out = model.generate(**inputs, max_new_tokens=128, do_sample=False)
answer = tokenizer.decode(out[0][inputs["input_ids"].shape[1]:],
                          skip_special_tokens=True)
```

token

len

```
if __【35】__(answer) > 128:      # 字符数校验
    answer = answer[:128]
```

题目5.5　多轮对话中 HISTORY[sid] 每轮写入两条消息（user 提问＋assistant 回复）。若代码只保留最近 __【36】__ 条消息，即保留最近 2 轮上下文（填数字）。

4

```
MAX_HISTORY_MESSAGES = __【36】__        # 保留最近 2 轮
recent = history[-MAX_HISTORY_MESSAGES:]
```

任务6：政策知识接入与检索增强（RAG）

题目6.1　对 docs/ 下 3 篇政策文档做切分，按自然段落切块且每块不超过 500 字。请补充默认参数与切分方式。

500

'\n\n'

```
def split_doc(text: str, max_len=__【37】__):
    blocks = text.split(__【38】__)      # 按自然段落切分
    ...
```

题目6.2　使用赛题指定中文嵌入模型向量化并建立 FAISS 索引。请补充模型 ID、归一化方法调用与索引维度。

BAAI/bge-small-zh-v1.5　模型ID:组织/模型名

512　这个估计就要现读了

```
from sentence_transformers import SentenceTransformer
import faiss
emb = SentenceTransformer("__【39】__")
vecs = emb.encode(chunks)
faiss.__【40】__(vecs)                  # 归一化，使内积等价余弦相似度
index = faiss.IndexFlatIP(__【41】__)  # bge-small-zh-v1.5 向量维度
index.add(vecs)
```

题目6.3　实现检索增强问答。请补全检索块数（本题指定 3 个），并写出该技术的英文简称（三个字母，即“检索增强生成”）。

3

RAG

```
def rag_answer(question: str, top_k=__【42】__):
    scores, ids = index.search(qvec, top_k)
    contexts = [chunks[i]["text"] for i in ids[0]]
    ...   # 检索结果拼入提示词再交由模型生成，该技术简称 __【43】__
```

题目6.4　持久化索引与切分块，文件名按提交规范命名。请补充索引文件名；并在保存前用断言保证“向量条数＝切分块数”，该内建属性名是 index.__【45】__。

policy.index

```
faiss.write_index(index, "submission/m2/rag/__【44】__")
assert index.__【45】__ == len(chunks), "向量条数与切分块数不一致"
```

ntotal

题目6.5　要使测试问“借阅卡补办需要什么材料？”的检索回答同时含“学生证”与“照片”两个关键词，检索应命中的知识文档是 docs/ 目录下的__【46】__（文件名含扩展名）。

这个真的要现场看,记住扩展名,比如.md

```
question = "借阅卡补办需要什么材料？"
answer = rag_answer(question)   # 命中文档：docs/__【46】__
```

题目6.6　超纲问题（如“明天天气怎么样？”）应友好拒答：当检索最高得分低于阈值时直接返回拒答文案。请补充判断条件中的比较运算符，以及加载已持久化索引的方法名。

<

```
threshold = 0.5
best_score = float(scores[0][0])
if best_score __【47】__ threshold:      # 低于阈值则拒答
    return {"answer": "未检索到相关政策资料，请咨询图书馆总服务台。"}
...
index = faiss.__【48】__("submission/m2/rag/policy.index")
```

read_index

任务7：应用性能与用户体验优化

题目7.1　为高频重复问题增加缓存。请补充哈希方法（本题指定 md5）、缓存有效期（秒）与写入缓存时使用的键。

md5

```
import hashlib, time
CACHE = {}
key = hashlib.__【49】__(question.encode("utf-8")).hexdigest()
if key in CACHE and time.time() - CACHE[key][0] < __【50】__:   # 有效期
    return CACHE[key][1]
...
CACHE[__【51】__] = (time.time(), answer)
```

300(按实际要求来)

key

题目7.2　模型服务异常时返回友好降级提示，返回体中用__【52】__字段标记本次为降级回复（本题指定该字段名）；同时保持 HTTP 状态码为__【53】__。

```
except Exception:
    return JSONResponse(
        content={"reply": "系统繁忙，请稍后重试，或致电图书馆总服务台6600。",
```

```
                "__【52】__": True},
        status_code=__【53】__) 200
```

fallback 非固定,只是为了统一后面的

题目7.3　运行性能对比脚本（附件 perf_check.py），结果中衡量优化后平均响应时间的字段是__【54】__；衡量优化前平均响应时间的字段是__【55】__。

avg_ms_off

```
python perf_check.py --n 15 | tee submission/m2/perf_compare.json
# 输出示例：{"avg_ms_off": 41.2, "__【54】__": 18.7, "n": 15}
```

avg_ms_on

题目7.4　perf_check.py 的 on 组对同一问题连续请求 n 次以验证缓存命中：第 1 次未命中，从第__【56】__次起全部命中缓存（填数字）。另：该脚本命令行参数 --n 的默认取值为__【57】__（填数字，见附件 argparse 配置）。

2

????

看文件

```
on = [timed_rag("借阅卡补办需要什么材料？") for _ in range(args.n)]
```

题目7.5　降级响应既要有降级标记，又要保持 HTTP 状态码 200，可借助 FastAPI 显式构造响应对象：`return __【58】__(content={...}, status_code=200)`。请补充类名。

JSONResponse

```
from fastapi.responses import __【58】__
return __【58】__(content={"reply": "...", "fallback": True}, status_code=200)
```

题目7.6　为便于核对摘要长度与显示缓存状态，请补充返回体字段名：摘要字符数用__【59】__字段；本次是否命中缓存用__【60】__字段；检索到的相似度列表用__【61】__字段（见应用实现）。

lenth

这几个要看实际比赛应用

```
return JSONResponse({"summary": answer, "__【59】__": len(answer)})
...
return JSONResponse({... "__【60】__": True})        # 命中缓存
...
return JSONResponse({... "__【61】__": [round(s, 4) for s in scores]})
```

CAKHE

scores

模块三：智能体层：多智能体编排与安全管控

任务8：运维工单智能体角色设计

题目8.1　工单流程为“读者报障→意图解析与规划→工具执行→质检复核→输出建议”。三个角色分工如下表，请补全角色英文名与诨名；此外设计文档还须为每个角色写明输入与__【64】__两类字段约定，以保证角色间 I/O 契约完整。

输出

```
| 角色 | 诨名 | 负责子任务 |
|---|---|---|
| __【62】__ | 调度员 | 意图解析与任务规划，输出步骤列表 |
| Executor | __【63】__ | 按计划调用工具并执行步骤 |
| QA | 质检员 | 对建议做质检与合规复核，并面向用户输出 |
```

Planner

维修工

题目8.2　角色间 I/O 约定采用 JSON 契约。请补充 intent 取值、steps 中表示工具名的字段名，以及执行日志必须记录的第三个字段名。

```
{"ticket_id": "T2001",
 "intent": "__【65】__", "steps": [{"__【66】__": "shelf_query", "args": {"room": "R305"}}]}

# Executor 每次工具调用须记录三个字段：
# {"tool": "shelf_query", "args": {...}, "__【67】__": {...}}
```

工单号

工作类型(意图)

工作步骤

ac_alarm

看角色的提示词或设计文档里怎么写的,这里应该是空调告警

tool

result

题目8.3　请把三角色设计与输入/输出字段约定写入设计文档 agent_design.md。设计文档须写出三角色的英文名与分工。其中负责“对建议做质检与合规复核，并面向用户输出”的角色英文名是__【68】__；三行对应的中文诨名依次为“调度员、__【69】__、质检员”。

```
# 角色设计（节选）
| 角色 | 诨名 |
|---|---|
| Planner | 调度员 |
| Executor | 维修工 |
| __【68】__ | __【69】__ |
```

QA

维修工

任务9：多工具驱动的智能体流程实现

题目9.1　在智能体中注册工具并封装调用。请补充工具名（URL 见附件 kb_tools_server.py）、HTTP 方法与超时秒数（本题指定 5 秒）。

```
TOOLS = {
    "shelf_query": "http://127.0.0.1:9500/tools/shelf_query",
    "__【70】__": "http://127.0.0.1:9500/tools/log_query",
    "kb_query": "http://127.0.0.1:9500/tools/kb_query",
    "resource_query": "http://127.0.0.1:9500/tools/resource_query"}

def call_tool(name, args):
    return requests.__【71】__(TOOLS[name], params=args, timeout=__【72】__).json()
```

log_query

get

5

题目9.2　实现“报障→规划→工具执行→质检→建议”完整流程。请补充：Planner 约束模型只输出 JSON 所用的角色名、解析 JSON 字符串的方法名、质检复核函数名（本题指定为 qa_check）。

system

```
messages = [{"role": "__【73】__", "content": PLANNER_SYSTEM},
            {"role": "user", "content": ticket}]
plan = llm_generate(messages)
steps = json.__【74】__(plan)["steps"] if isinstance(plan, str) else plan["steps"]
results = [call_tool(s["tool"], s["args"]) for s in steps]
draft = executor_summarize(steps, results)
final = __【75】__(draft)
```

loads

qa_check

题目9.3　空调告警工单的规划顺序为“先确认设备状态、后查询服务日志”。第二步查询日志的工具为 log_query（URL 见题目9.1），则第一步确认设备状态的工具是__【76】__（工具名）。

```
# 规划顺序：先确认，后查日志
steps = [{"tool": "__【76】__", "args": {"room": "R305"}},
         {"tool": "log_query", "args": {"service": "rfid"}}]
```

shelf_query

题目9.4　运行日志采用追加写、逐行记录。请补充打开文件时的模式参数（单字符），以及把字典序列化为 JSON 字符串的方法名（注意与题目9.2 的解析方法区分）。

a

```
f = open("submission/m3/agent_run_log.jsonl", "__【77】__", encoding="utf-8")
line = json.__【78】__({"tool": name, "args": args, "result": result},
                       ensure_ascii=False) + "\n"
f.write(line)
```

dumps

题目9.5　请阅读附件 kb_tools_server.py 源码回答问题（答案以附件为准）。

```
# 附件源码节选
def log_query(service: str = Query("__【79】__")):            # 1) 默认服务代号
    return {"service": service, "log": "..."}

def shelf_query(room: str = Query("R305")):
    return {"room": room, "status": "空调停机", "level": __【80】__}    # 2) 告警级
别

def kb_query(q: str = Query("借阅卡补办")):
    return {"q": q, "result": "命中: ...维修/制卡人员2个工作日内__【81】__处理。"}  #
3) 处置词

@app.get("/tools/__【82】__")                                  # 4) 第 4 个工具（电
子资源并发状态查询）
```

resource _query

```
def __【82】__(db: str = Query("cnki")):
    return {"db": db, "concurrent": 128, "limit": 120, "status": "已满"}
```

任务10：跨场景编排与安全管控

题目10.1　基于关键词的规则路由：命中运维/设施关键词进“设备报修智能体”（本题指定标识符 repair_agent），否则进“访客咨询智能体”（本题指定标识符 visitor_agent）。请补充标识符、路由方式（四个中文，见题面括号提示）与统一入口用于标明实际应答者的字段名（本题指定）。

```
REPAIR_KEYWORDS = ["报修", "故障", "空调", "门禁", "网络"]

def route(question: str) -> str:
    if any(k in question for k in REPAIR_KEYWORDS):
        return "__【83】__"
    return "__【84】__"

# 该路由方式属于 __【85】__ 路由
# 统一入口返回结果中用 __【86】__ 字段标明实际应答的智能体
```

repair_agent
visitor_agent
规则
'agent_name'
( )

题目10.2　敏感指令拦截。请补充拦截文案、标记字段的值，以及该机制所属的安全防护层级（两字中文）。

```
SENSITIVE = ["删除数据库", "关机", "重启服务器", "清除日志"]

if any(s in question for s in SENSITIVE):
    return {"reply": "__【87】__", "blocked": __【88】__}
# 该机制属于 __【89】__ 安全防护（拦截事件须写入审计日志）
```

True

题目10.3　权限校验：重启设备、关闭工单为高权限操作，仅管理员可执行。请补充角色名、无权限时的状态码，以及该校验所属的控制类型（两字中文）。

```
role = request.headers.get("X-User-Role", "user")
if action in ["restart", "close_ticket"] and role != "__【90】__":
    raise HTTPException(status_code=__【91】__, detail="权限不足")
# 该校验属于 __【92】__ 控制
```

admin( )
403
/

题目10.4　操作审计采用 JSONL 格式，每条记录含四个字段（本题指定字段集为 timestamp、user、action、result）。其中记录“操作发生时间”的字段名是__【93】__；记录“操作者身份”的字段名是__【94】__。

```
record = {"__【93】__": now_iso(), "__【94】__": user, "action": action, "result": result}
```

timestamp
user

```
    append_jsonl(AUDIT_LOG_PATH, record)
```
